

Debugging Challenge Pack (5 Bugs) — Questions Only

Author: Sai Mahendra

Repo: forage-midas (Debug branch)



JIRA Bug Ticket (Challenge #1)



JIRA-101: Daily Credit Total fails with IndexOutOfBoundsException

Complexity: EASY

Component: TransactionProcessor.java

Description:

Daily credit total calculation crashes while iterating through filtered transactions. The method filters correctly, but fails during summation.

Steps to Reproduce:

1. Run TransactionProcessorTest.
2. Test calls calculateDailyCreditTotal() with 3 matching CREDIT transactions.

Actual Result:

IndexOutOfBoundsException (attempts to access an invalid index while looping).

Expected Result:

Should return correct daily CREDIT total (example expected total: 425.75) without crashing.



JIRA Bug Ticket (Challenge #2)



JIRA-102: Validation method mutates original Account objects (side effect bug)

Complexity: HARD

Component: TransactionValidator.java

Description:

findInsufficientFundAccounts() is intended to simulate transfers and report which accounts would go negative. But after calling it, the original Account objects passed into the method have their balances changed.

Steps to Reproduce:

1. Run TransactionValidatorTest.
2. Observe that "original balances unchanged" test fails after validation call.

Actual Result:

Original account balances are modified after validation runs.

Expected Result:

Validation should be side-effect free: it may simulate using working state, but the original input objects must remain unchanged.



JIRA Bug Ticket (Challenge #3)



JIRA-103: Interest Calculation Resulting in Zero due to Integer Division

Complexity: 🟡 MEDIUM

Component: InterestCalculator.java

Description:

The savings interest module is failing to calculate interest for any account where the annual rate is less than 12%. The calculation logic performs integer division on the interest rate, causing the rate to truncate to zero.

Steps to Reproduce:

1. Set an annual interest rate of 6%.
2. Calculate interest for any balance and any number of months.
3. Run InterestCalculatorTest.

Actual Result:

\$0.00 (monthly rate is effectively calculated using integer division like $6 / 12 \rightarrow 0$).

Expected Result:

Interest should be calculated based on the fractional monthly rate (e.g., 0.5% or 0.005).

Example:

- Balance 10000.00, 6% annual, 6 months -> 300.00
- Balance 1000.00, 3% annual, 1 month -> 2.50



JIRA Bug Ticket (Challenge #4)



JIRA-104: NullPointerException when user address list is null

Complexity: 🔴 HARD

Component: UserProfileService.java

Description:

getPrimaryAddress(User user) iterates over user addresses to find the primary address. For some users, the address list is null (new user / partial data), and the method crashes.

Steps to Reproduce:

1. Run UserProfileServiceTest.
2. Test passes user with addresses = null.
3. Call getPrimaryAddress(user).

Actual Result:

NullPointerException during iteration.

Expected Result:

Method should not crash. It should return null when addresses list is null or empty.



JIRA Bug Ticket (Challenge #5)



JIRA-105: Vanishing money during concurrent deposits (race condition)

Complexity: 🟡 MEDIUM

Component: ConcurrentBankAccount.java

Description:

When multiple threads deposit into the same account, the final balance sometimes ends up smaller than expected. No exception is thrown; money silently disappears.

Steps to Reproduce:

1. Run ConcurrentBankAccountTest (multi-threaded test).
2. Repeat test a few times if needed.

Actual Result:

Final balance < expected total deposits (lost updates).

Expected Result:

Final balance must always equal expected total deposits. Deposit operation should be thread-safe.